

NSS — Milník 1: Architektonická Analýza

Projekt: Football Manager Bot

Autor: Yernur Bauyrzhanuly, bauyryer

Gitlab: <https://gitlab.fel.cvut.cz/bauyryer/football-bot-nss>

1. Popis a motivace

Football Manager Bot je platforma pro správu amatérských fotbalových zápasů.

Skládá se z Telegram Bota (pro hráče) a webové aplikace WebApp (pro adminy).

Systém organizuje přihlášky, automaticky vyvažuje týmy podle ELO ratingu hráčů a eviduje výsledky.

Motivace: Organizace her přes WhatsApp nebo Telegram skupinu je ruční a náchylná k chybám. Bot tuto práci automatizuje. Projekt reálně používá skupina 45 hráčů, s plánem rozšíření na více nezávislých skupin (Multi-Tenant SaaS).

Poznámka: Projekt existoval před předmětem NSS jako funkční prototyp.

V rámci NSS provádíme formální analýzu AS-IS architektury a návrh přechodu na modulárnější strukturu (TO-BE).

2. Funkční požadavky

- **FR1** — Admin vytvoří zápas (datum, místo, kapacita, cena).
- **FR2** — Hráč se přihlásí nebo odhlásí přes Telegram.
- **FR3** — Systém spravuje listinu náhradníků automaticky.
- **FR4** — Bot vyvažuje týmy podle ELO a herní pozice.
- **FR5** — Hráči hlasují o MVP po zápase.
- **FR6** — Bot přepočítá rating hráčů po uzavření zápasu.
- **FR7** — Admin spravuje soupisku ve WebApp (kick, guest, rezerva).
- **FR8** — Systém izoluje data různých skupin (multi-tenancy).
- **FR9** — Bot posílá automatické připomínky před zápasem.
- **FR10** — Hráč vidí osobní statistiky a historie zápasů.

3. Nefunkční požadavky

- **NFR1** — Odezva na akci hráče do 2 sekund.
 - **NFR2** — Kritické operace jsou transakční (SELECT FOR UPDATE).
 - **NFR3** — Nasazení na vlastní server, dostupnost 24/7.
 - **NFR4** — Automatické nasazení přes GitHub Actions + Docker + Watchtower.
 - **NFR5** — WebApp je zabezpečena HMAC validací Telegram initData.
 - **NFR6** — Admin práva jsou ověřována middleware vrstvou.
 - **NFR7** — Databázové schéma je verzováno pomocí Alembic migrací.
-

4. Uživatelé

- **Hráč** — přihlašuje se, hlasuje, prohlíží statistiky.
 - **Admin skupiny** — vytváří a spravuje zápasy v jedné skupině.
 - **System Owner** — technický správce, řeší systémové operace přes skupinami.
-

5. Případy užití

UC1: Hráč se přihlásí na zápas

Hráč klikne na tlačítko v Telegramu. Systém ověří kapacitu pomocí transakčního zámku (SELECT FOR UPDATE). Pokud je místo, zapíše hráče. Pokud ne, jde do rezervy. Zpráva v chatu se okamžitě aktualizuje.

UC2: Admin vytvoří zápas

Admin vyplní formulář ve WebApp. Facade (GameLifecycleService) vytvoří zápas v DB, naplánuje připomínky v Scheduleru a odešle zprávu s tlačítky do Telegram skupiny.

UC3: Vyvažování týmů

Admin spustí balancování. Systém seřadí hráče podle ELO a pozice a rozdělí je do dvou vyrovnaných týmů (Strategy Pattern).

UC4: Hlasování o MVP

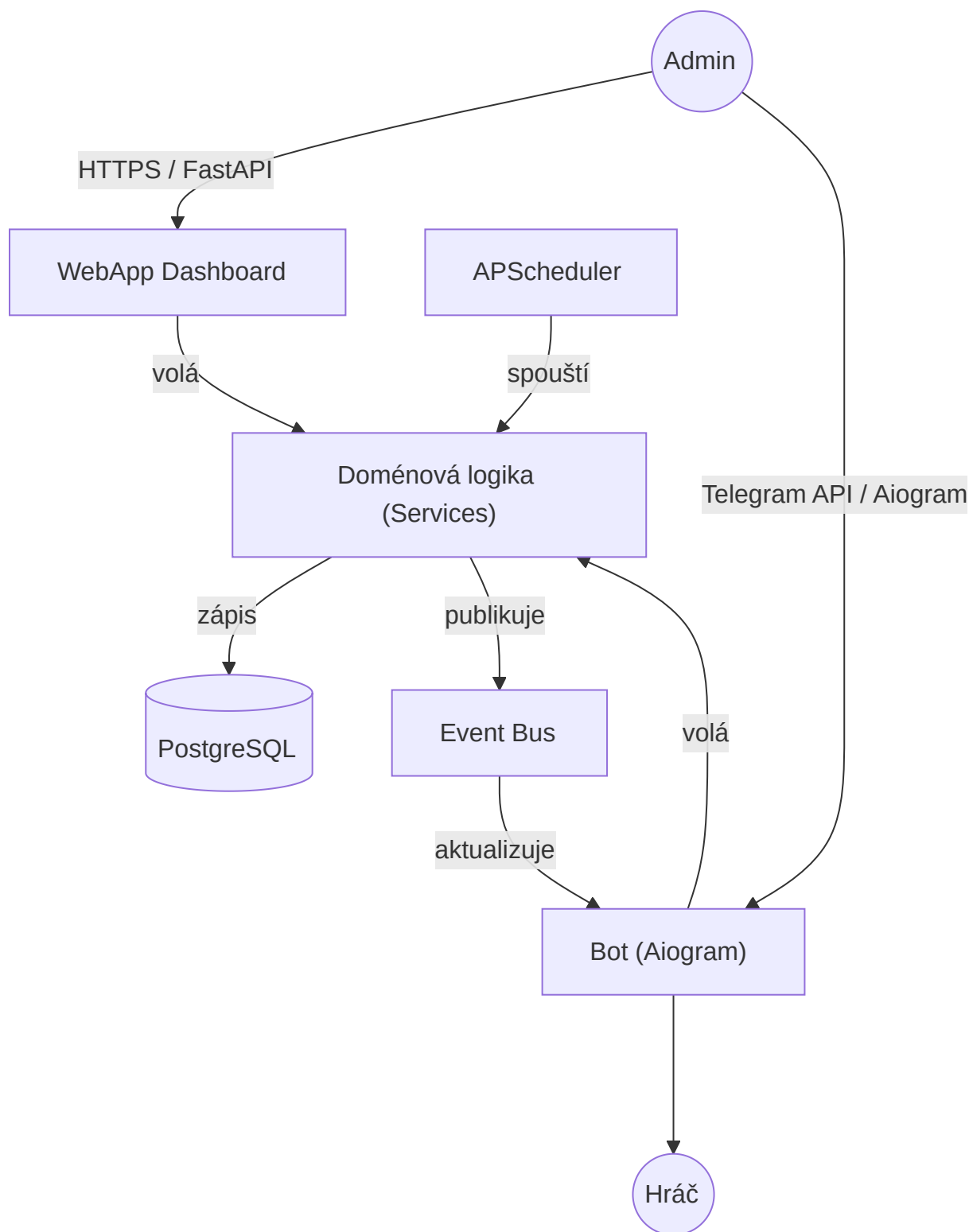
Scheduler automaticky spustí hlasování po zápase. Hráči hlasují přes Telegram.

Po vypršení bot vyhodnotí výsledky a aktualizuje ELO a statistiky.

UC5: System Owner sloučí duplicitní uživatele

Owner zavolá `/merge_users <old_id> <new_id>`. Systém přepíše všechny záznamy (Signup, Vote, RatingHistory) na nový účet a smaže starý profil.

6. Jak systém vypadá na první pohled



7. Struktura projektu (AS-IS)

```
app/

├─ api/ # REST API (FastAPI)

|   └─ routers/

|   └─ admin.py # 475 řádků – God Router, duplicitní logika ← PROBLÉM

|   └─ games.py

|   └─ users.py

|   └─ voting.py

├─ bot/ # Telegram Bot (Aiogram 3.x)

|   └─ admin_handlers.py # 398 řádků – Leaky Abstraction ← PROBLÉM

|   └─ admin_system.py # 274 řádků

|   └─ admin_dashboard.py # 231 řádků

|   └─ game_handlers.py # Správný vzor (TO-BE příklad)

|   └─ listeners.py # Dual Session, nekonzistence ← PROBLÉM

|   └─ middlewares.py # Validate a Session

├─ core/ # Business logika

|   └─ services/

|       └─ roster.py # 258 řádků – Hardcoded business rules ← PROBLÉM

|       └─ stats.py # Inkonzistence ratingů ← PROBLÉM

|       └─ game_lifecycle.py

|   └─ events.py # Volatile EventBus (RAM only) ← PROBLÉM

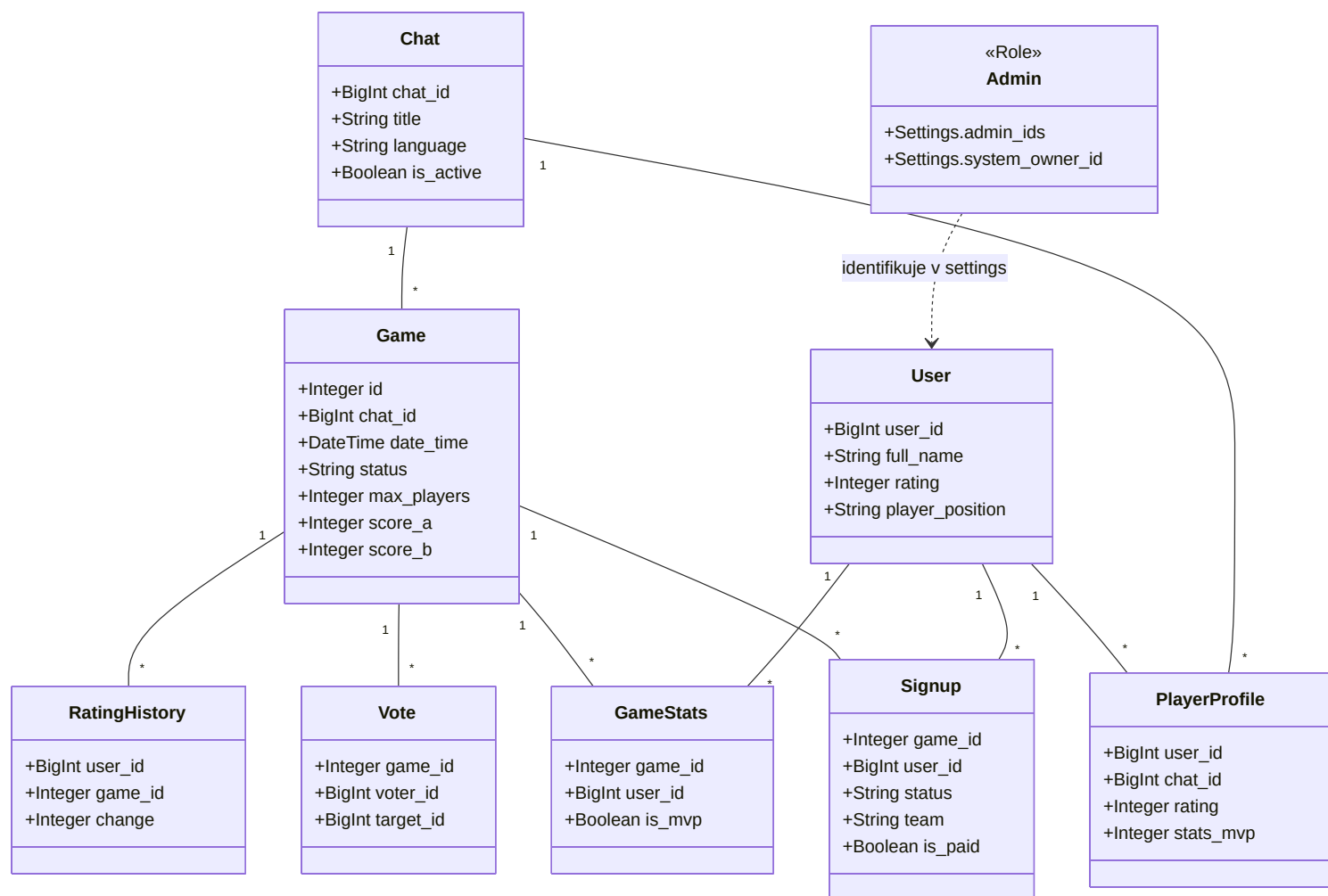
└─ db/

    └─ models.py # 221 řádků, 9 tříd – God Object ← PROBLÉM
```

8. Problémy (AS-IS analýza)

8.1 Celkový doménový model (Class Diagram)

Tento diagram popisuje kompletní strukturu dat a vztahů v systému. Ukazuje, jak jsou entity propojeny a jak je řešena role Admina (pouze přes externí nastavení).



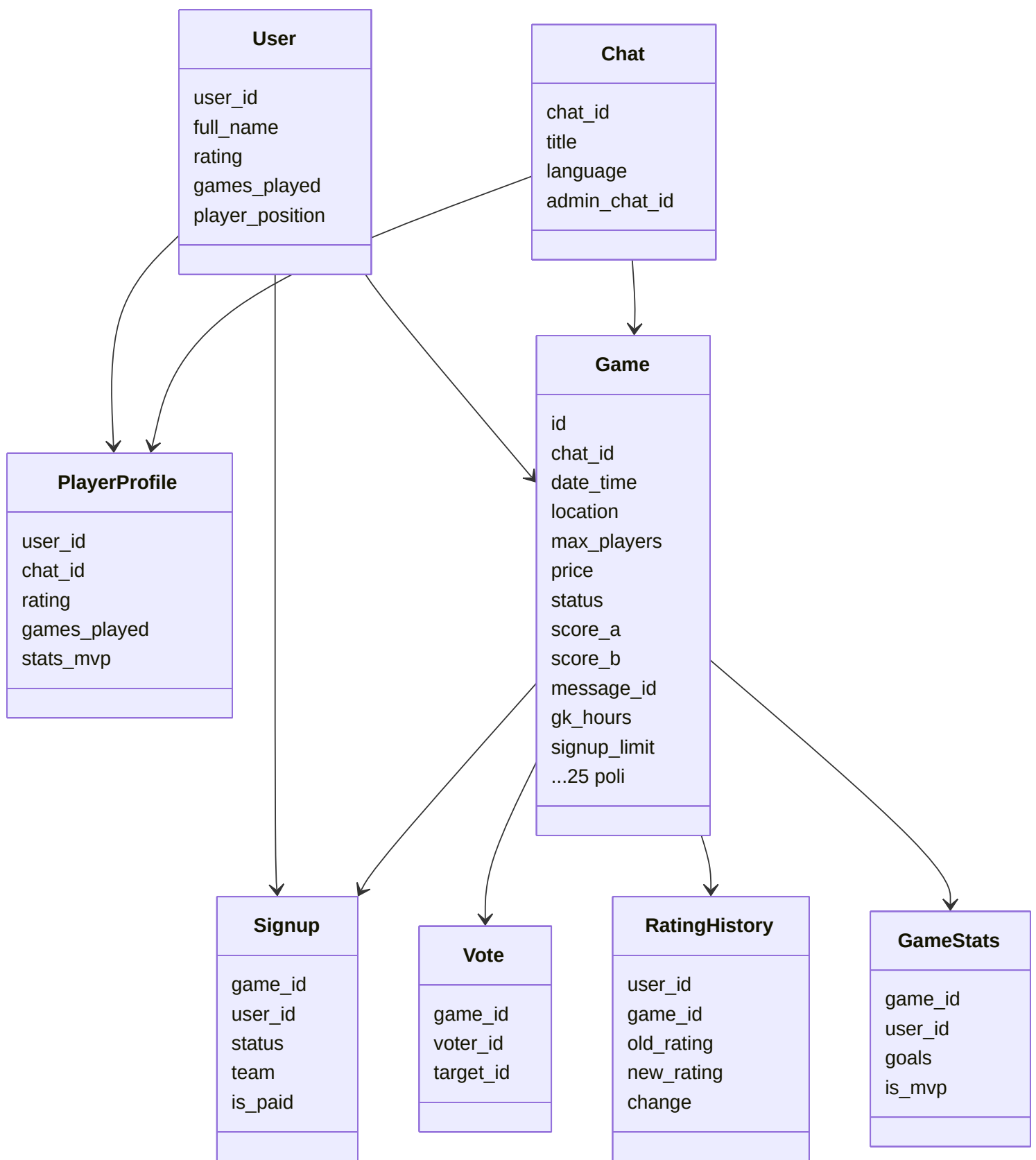
Poznámka: Role Admina není v databázi uložena jako entita, což je jedna z hlavních architektonických slabin — kontrola práv probíhá proti statickým seznamům v konfiguraci.

8.2 Detailní analýza: God Object — models.py

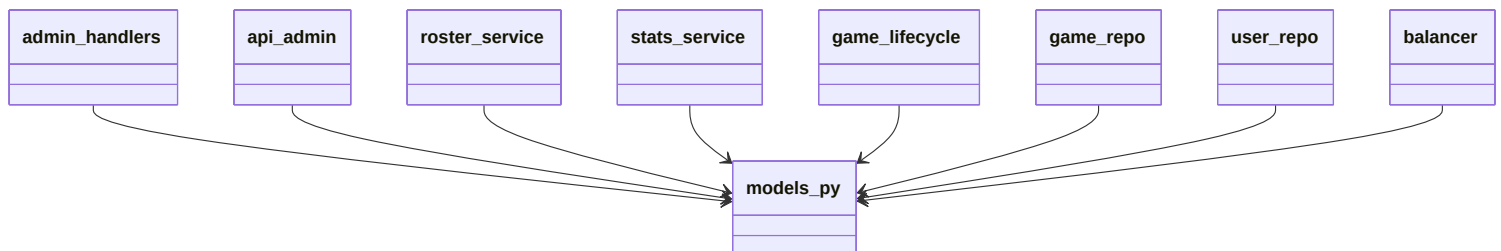
Veškerý datový model systému (9 entit) leží v jediném souboru 221 řádků.

Entita **Game** má přes 25 polí — obsahuje geografické, finanční, herní i technické

informace najednou. Jakákoliv změna v tomto souboru vyžaduje kontrolu celého projektu.



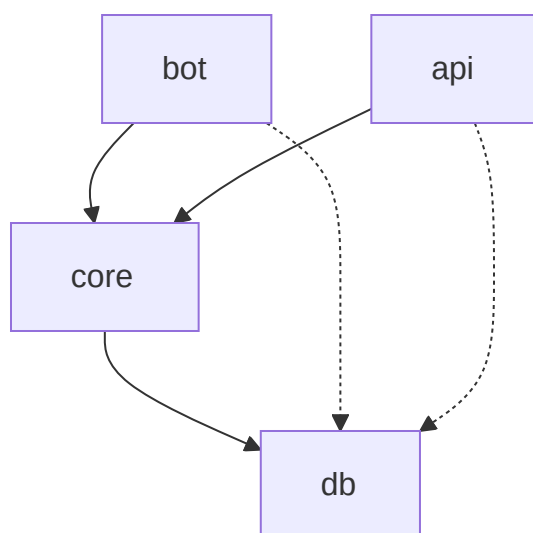
Problém s vazbami: Každý modul v projektu importuje přímo z `models.py`:



Problém 2: Protékající vrstvy (Layer Violation)

Bot vrstva by měla komunikovat jen přes Core služby. Ve skutečnosti

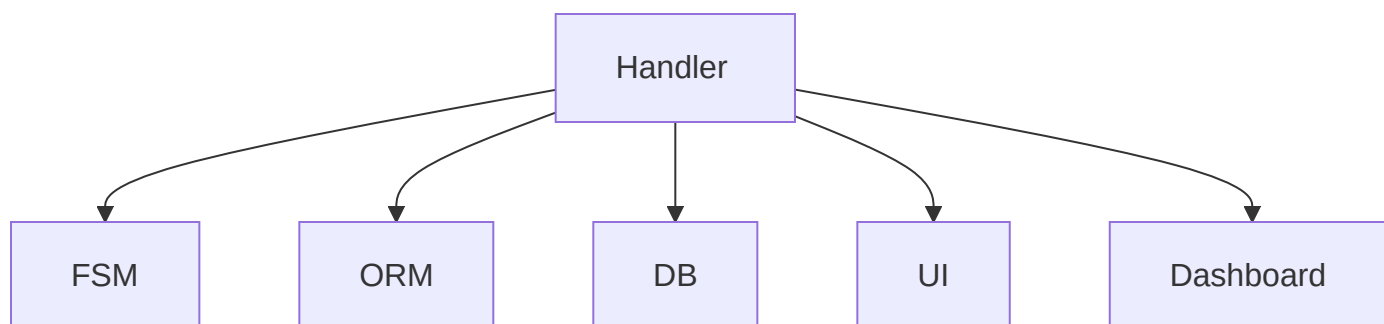
importuje přímo modely z DB vrstvy:



Plné čáry = správně. Čárkované = přímý import models.py (Layer Violation).

Problém 3: Leaky Abstraction v Bot handleru

Handler `process_guest_position` dělá příliš mnoho věcí najednou:

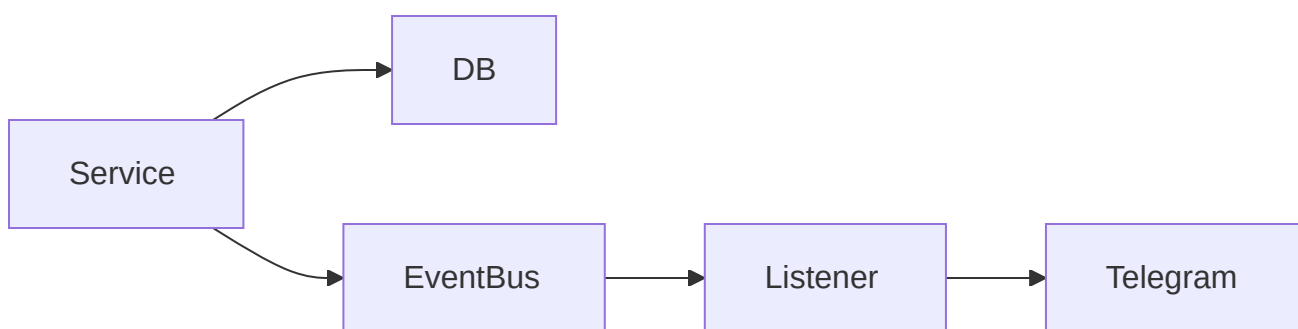


Jeden handler čte FSM stav, vytváří ORM entity, řídí transakci, formátuje HTML

a aktualizuje dashboard. Toto porušuje SRP a neumožňuje znovupoužití kódu.

Problém 4: Volatile Event Bus

EventBus existuje pouze v paměti (RAM). Je to globální singleton.

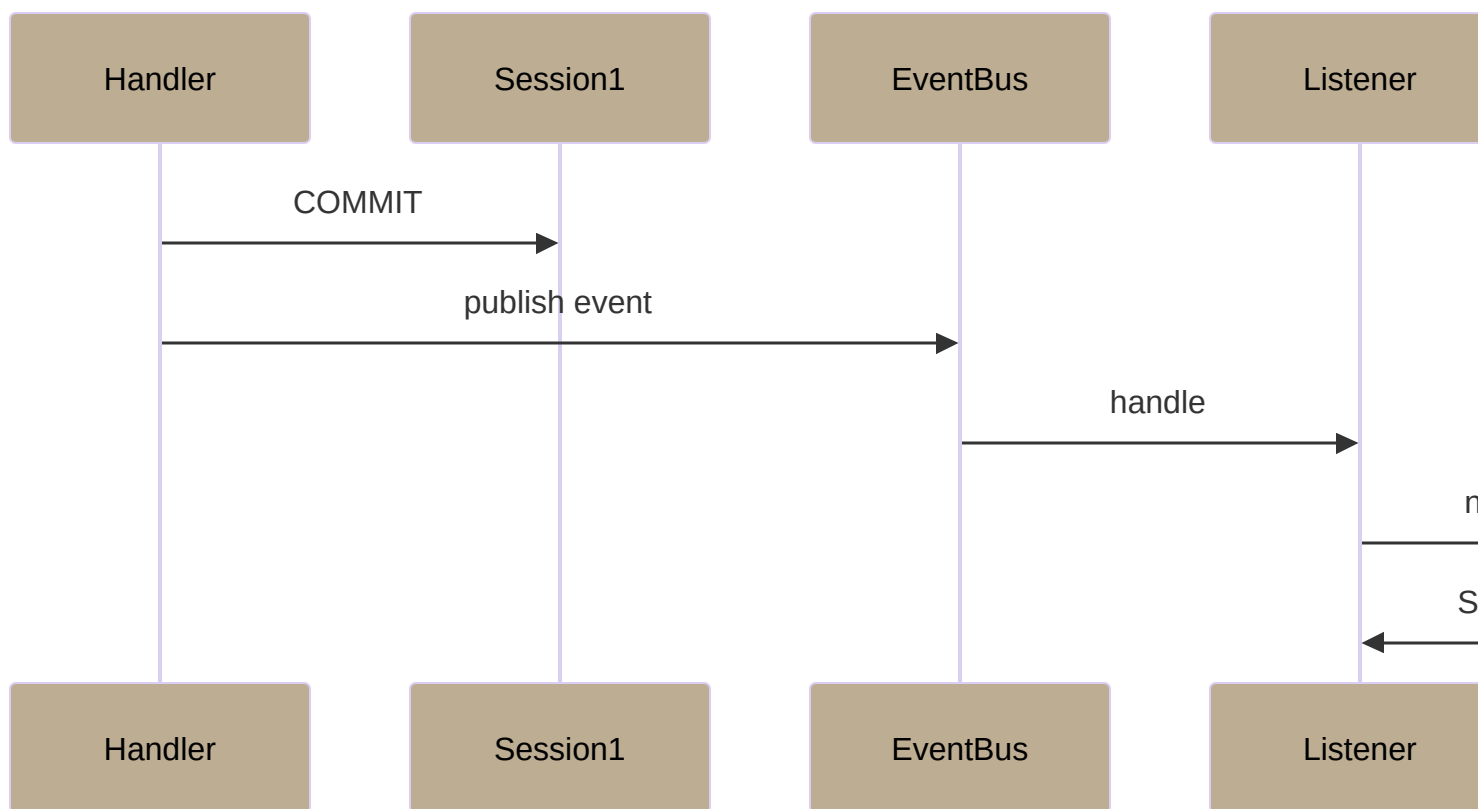


Riziko: Při restartu procesu se ztratí všechny čekající eventy.

Notifikace se nikdy nedostanou k uživatelům.

Problém 5: Dual Session (transakční nekonzistence)

Handler a Listener používají dvě různé databázové sessions:



Pokud Listener selže (timeout Telegramu) — data jsou uložena v DB,

ale UI v Telegramu se neaktualizuje. Není žádný mechanismus retry.

Problém 6: Duplicitní logika — add_guest

Stejná operace je implementována na dvou místech s různým výsledkem:

```
bot/admin_handlers.py:370 → status = ACTIVE (hráč rovnou hraje)
```

```
api/routers/admin.py:446 → status = RESERVE (hráč jde do rezervy)
```

Záleží na tom, odkud admin hosta přidá — výsledek je jiný.

Problém 7: Multi-Tenancy — globální konstanty

Business pravidla jsou napevno zakódovány pro všechny skupiny:

| Pravidlo | Kde | Problem |

|-----|-----|-----|

| 36h odhlaseni | roster.py r.117 | Skupina A chce 24h |

| GK priorita 48h | roster.py r.87 | Nelze vypnout |

| User.rating vs PlayerProfile.rating | stats.py r.153 | Data jdou obemat smerema |

Problém 8: API importuje Bot singleton

```
api/auth.py → from app.bot.instance import bot
```

```
api/routers/admin → from app.bot.instance import bot
```

```
api/routers/games → from app.bot.instance import bot
```

REST API nesmí záviset na Telegram Botu. Tato závislost znemožňuje

testovat API bez inicializace celého bota.

9. Výběr vhodné architektury (TO-BE návrh)

Pro další rozvoj projektu byl zvolen architektonický styl **Modulární Monolit (Vertical Slices)**. Tento výběr přímo reaguje na identifikované problémy AS-IS stavu.

Proč Monolit?

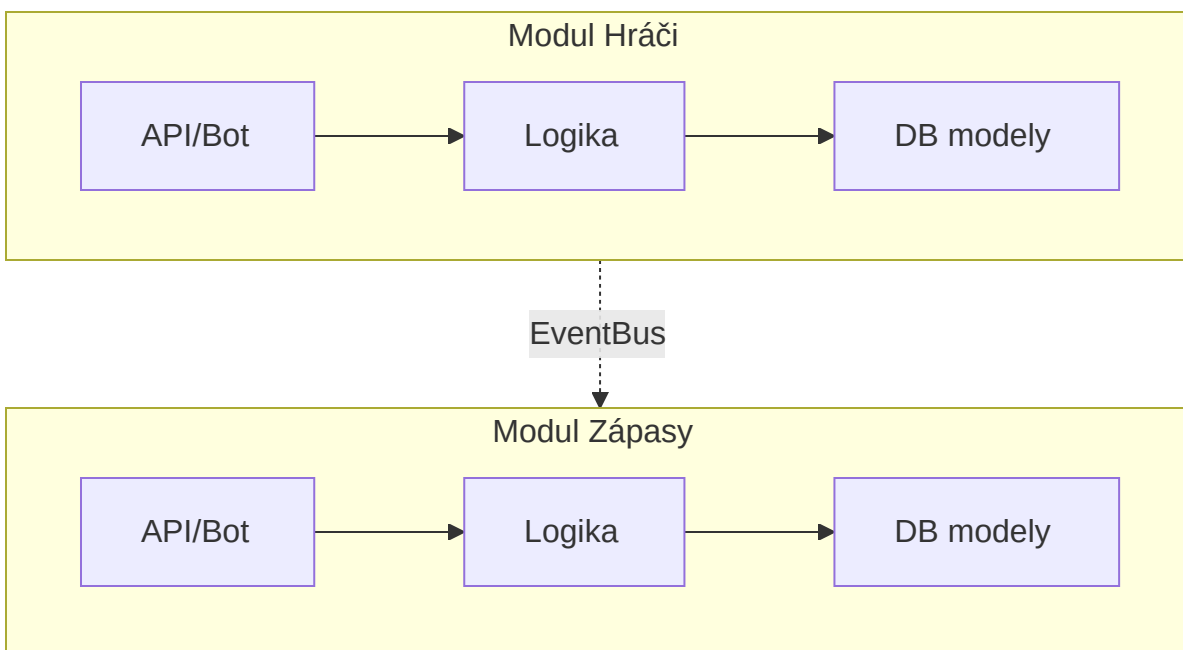
I při plánovaném škálování na stovky skupin a tisíce uživatelů zůstává monolit optimální volbou z několika důvodů:

1. **Telegram Best Practices:** Pro vývoj botů se (i při větším zatížení) doporučuje monolitická struktura. Důvodem je potřeba minimální latence při zpracování asynchronních webhooků a jednodušší správa stavu konverzací.
2. **Efektivita škálování:** 1000 uživatelů představuje pro moderní technologie (Python/FastAPI + PostgreSQL) minimální zátěž. Rozdělení na mikroslužby by v této fázi přineslo pouze infrastrukturní režii bez reálného přínosu pro výkon.
3. **Multi-tenancy:** Izolace dat pro stovky nezávislých skupin je v rámci jednoho monolitu (pomocí `chat_id`) mnohem přímočařejší. Mikroslužby by vyžadovaly složitou orchestraci a sdílení identit uživatelů mezi jednotlivými službami.

Co jsou Vertical slices?

Namísto klasického dělení na "technické vrstvy" (kde jsou všechny modely v jedné složce a všechny handlers v druhé), přecházíme na dělení podle **business funkcí**.

- **Horizontální vrstvy (AS-IS):** Při změně pole v zápase musíme změnit 5 různých složek (`api/`, `bot/`, `core/`, `db/`). Kód je rozprostřen po celém projektu.
- **Vertikální řezy (TO-BE):** Modul `Matches` obsahuje vše, co se týká zápasu: DB model, logiku pro vyvážení i Telegram handlers.
- **Výhoda:** Vysoká soudržnost (**High Cohesion**). Věci, které se mění společně, jsou v kódu u sebe. Moduly spolu komunikují pouze přes definovaná rozhraní nebo asynchronní eventy, což minimalizuje závislosti (**Low Coupling**).



10. Návrh řešení: Problém → Vzor

V rámci druhého milníku navrhujeme vyřešit identifikovaný technický dluh pomocí následujících vzorů:

| Problém | Navrhovaný vzor | Účel |

|-----|-----|-----|

| **God Object** | Vertical Slices | Rozbití obřích modelů na malé, doménově specifické entity. |

| **Layer Violation** | Repository | Odstínění business logiky od přímého volání databáze. |

| **Dual Session** | Unit of Work | Zajištění, že se všechna data v rámci požadavku uloží jako jedna transakce. |

| **Volatile Event Bus** | Transactional Outbox | Ukládání událostí do DB/Redis, aby se neztratily při restartu bota. |

| **Duplicitní logika** | Facade / Service Layer | Sjednocení operací (např. add_guest) do jediné společné služby. |

| **Admin validace** | Chain of Responsibility | Přesun kontrol práv do middleware vrstvy namísto opakování v každém handleru. |

| **Hardcoded rules** | Strategy | Umožnění změny herních pravidel (např. čas odhlášení) pro každou skupinu zvlášť. |